Q7: Intel Image Classification using Transfer Learning (PyTorch)

Objective

Build an image classification model to classify outdoor scene images into the following classes:

1. Buildings
2. Forest
3. Glacier
4. Mountain
5. Sea
6. Street

Dataset: https://www.kaggle.com/datasets/puneet6060/intel-image-classification

Part A: Data Preparation (5 Marks)

Step 1: Install Kaggle and Download Dataset

```
!pip install kaggle
```

```
Requirement already satisfied: kaggle in /usr/local/lib/python3.12/dist-packages (2.0.2)
Requirement already satisfied: bleach in /usr/local/lib/python3.12/dist-packages (from kaggle) (6.4.0)
Requirement already satisfied: jupytext in /usr/local/lib/python3.12/dist-packages (from kaggle) (1.19.3)
Requirement already satisfied: kagglesdk<1.0,>=0.1.20 in /usr/local/lib/python3.12/dist-packages (from kaggle) (0.1.23)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from kaggle) (26.2)
Requirement already satisfied: protobuf in /usr/local/lib/python3.12/dist-packages (from kaggle) (5.29.6)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.9.0.post0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.12/dist-packages (from kaggle) (8.0.4)
Requirement already satisfied: requests in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.32.4)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from kaggle) (4.67.3)
Requirement already satisfied: urllib3>=1.15.1 in /usr/local/lib/python3.12/dist-packages (from kaggle) (2.5.0)
Requirement already satisfied: webencodings in /usr/local/lib/python3.12/dist-packages (from bleach->kaggle) (0.5.1)
Requirement already satisfied: markdown-it-py>=1.0 in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (4.2.0)
Requirement already satisfied: mdit-py-plugins in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (0.6.1)
Requirement already satisfied: nbformat in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (5.10.4)
Requirement already satisfied: pyyaml in /usr/local/lib/python3.12/dist-packages (from jupytext->kaggle) (6.0.3)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil->kaggle) (1.17.0)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.12/dist-packages (from python-slugify->kaggle) (1.3
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.4.
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (3.18)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests->kaggle) (2026.5.20)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=1.0->jupytext->kaggle
Requirement already satisfied: fastjsonschema>=2.15 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kaggle)
Requirement already satisfied: jsonschema>=2.6 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kaggle) (4.2
Requirement already satisfied: jupyter-core!=5.0.*,>=4.12 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->k
Requirement already satisfied: traitlets>=5.1 in /usr/local/lib/python3.12/dist-packages (from nbformat->jupytext->kaggle) (5.7.
Requirement already satisfied: attrs>=22.2.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupytex
Requirement already satisfied: jsonschema-specifications>=2023.03.6 in /usr/local/lib/python3.12/dist-packages (from jsonschema>
Requirement already satisfied: referencing>=0.28.4 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->j
Requirement already satisfied: rpds-py>=0.25.0 in /usr/local/lib/python3.12/dist-packages (from jsonschema>=2.6->nbformat->jupyt
Requirement already satisfied: platformdirs>=2.5 in /usr/local/lib/python3.12/dist-packages (from jupyter-core!=5.0.*,>=4.12->nb
Requirement already satisfied: typing-extensions>=4.4.0 in /usr/local/lib/python3.12/dist-packages (from referencing>=0.28.4->js
```

```
from google.colab import files
files.upload()
```

Choose Files download.jpg

download.jpg(image/jpeg) - 9170 bytes, last modified: 6/27/2026 - 100% done

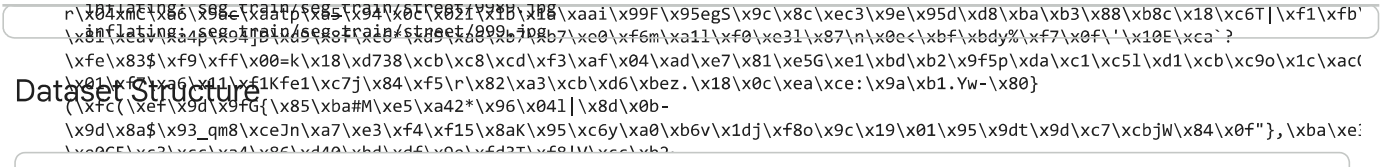
```
Saving download.jpg to download.jpg
{'download.jpg':
b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00\x00\x01\x00\x01\x00\x00\xff\xdb\x00\x84\x00\t\x06\x07\x08\x07\x06\t\x08\x07\x08'
\x1d"" \x1d\x1f\x1f$(4,$&1\'\x1f\x1f-=-157:::#+?D?
8C49:7\x01\n\n\n\r\x0c\r\x1a\x0f\x0f\x1a7%\x1f%7777777777777777777777777777777777777777777777777\xff\xc0\x00\x11\x08\x00\x94\
\x00\xcap\xa6\x84\xf6Y\xdagd\x94#\xe9\x04wd89\xdf\xe7M\xfb;\xc3Zn\x0fdm\xee\xa3<\xc8\x83H\xab>\x926\xfb|)}\xd5\x8d\xa5\x9fd\xe7
haM!K\xe5\x86\t\x18\xf5\xf3\xa1\xff\x00\xc3H\xb9\xab{\x1a\xca\xd0\xcb#\xaa\xa3\xa8\xd8t\xceG\xce\xae\xe2\xd6\xa9m\xc1\xee\x160
(\xc0v\'R\xea\xd8\x8c\x03\x8f\\\xe7\xa7\x99\xa5\x13^K\x04\xac\x85\xa4\x90\x88\xb4\xba.H\x03;\x01\xe1\xe5L\xf8K%\xcf\x0c\xe6^@\
j\xb6o\xc2>^\x1a[\x80
\x82\xe1\xccb\xe2GVE\xc8.\x06\xddGL~\xe2\xbec\xd9\xc5\xd7\xda;0\xa9\x93\xcc\xc6\x93\xe3\xdd5\xf4\x9e-;\xcbj&\xb8\x95\xc4\xec\x
\r\xcd|\xf3\xb2X\x93\xb5\x16\xba\xc6\x90X\x8d\xc7BA\xa1\x81\xbf\x8e@\x8f\r\x7f\t\xb4\x9axx\xc7>5E\x17\x8eu\x12L\x8c\xc0\x03\xe
\xe9\xb9\x03X\xf4\xdcb\x80\x8f\xda\x1f\x83E$$\xff\x00N\x14(z\xeeWs\xf1\xea>B\xaf\xed5\xc9\x9a\xc4\x85\x89\xd72\xb1\xd0\xcc\x18
\n>^\xff\x00\x13\x95Z\x15\x8b\x99\x04\x1f\xf5\x146\xad\x9bq\xe5\xd2\xb35\xd0V\x81-
\x1f]\xca\x80\x1a`\xf1\xb9]^{\x9f\x9f\xef]\xed7Ymo\x11\x8f1\x8as5\x11\x9c`\x9c\x9abdh$\x8aL\x8d@\xb3a\x97\x18\xc8\x14\xbb\x81\
```

```
!mkdir -p ~/.kaggle
!cp kaggle.json ~/.kaggle/
!chmod 600 ~/.kaggle/kaggle.json
```

```
\xe56\x1dg\xd4yU\xd6\xf3@gT\xe608\xd9\xd4\xec\xa7\xc7~\xa3\xce\xb28\xa7+\'\x1d\xb1\x17k\x19M\x8a\xa1\xd9\xc3\x922z\x83\xe1\xf7'
cp: cannot stat 'kaggle.json': No such file or directory
#\xe5\xf0n\x9f\x122\xbd\xa1_@j\x9f\xda\xb3v\xa5}\xa6\xf7\xbb\xddhI\xc2\x8ccqZN\x1fw
chmod: cannot access '/root/.kaggle/kaggle.json': No such file or directory
\xe0\xd1@\xdd\xe1\xca\xca0\xb60\xd9\xfc\xc5>QB,\xe1\x89\xa3\x13\x16\x8dXr\xc4h\xe3!\x98\xab\xe7\xcf`\x06jCw/\xf9\xe3I;+\x85\x8
\xe1\x81\x8f\x19x-
```

```
!kaggle datasets download -d puneet6060/intel-image-classification
!unzip intel-image-classification.zip
```

```
\xfd\xabqk\xaa+h\xd9\x168\xd8\xaa\x80\xc8\xe5s\xf1\x07j\xc7\x93\xf1\xa2\x16/\xed\x15\xd4\xb0\xda\xf1\x06e(E\xb9M
\x8c\x00v8\xfex\xd2\x8e\xc5\x10\xfc\x13\x8ae\xb05cm\x8e\xe0\n/\xb4/"Z_\xb2\x81\xcb(\x10\xea9\xc1\xd4?
\xbd%\xec\xcbg\x83_\xa6\x92\xba\xa7N\xf0\xf38\xaa%x\x99OF\xacq\x05\x92\xfb\x92"\x0cp\xa1\x9c\xa9f#|\x02\t\xf0\xc5Ym\xfe\x95\xa7
\x16\xc4\xc2F.\x02\xb2\xa0:\xb6\r\xb8\xf8`Qv\x88\x82k\x9dr\xb6\x94\x93\x1a\xc0\xcb\r\x8621Y\\ih\x9ba\xbd\xb2\xbd\x92N\xc9[%\xca
(]\xbc\x7f\xb5[\xc2\xdc{]\xcc\xac\x81Yn\x86\x03\xe7\x04\x81\xd7c\xd2\xa9\x8eY\x1e\xfa97\xcb\t\t8\x1e~
WWG|4Q\xb5\xc5\xcd\xbf\xff\x00\xb3\xcbE\x00\x9dm\xb2\x9a\xaa\xc8\xdc\xc9w
\x04I\xaf\xde|\x81\xe03\xe5\xe5C#\xcc\xf6\x99|\xe5\x10w\x80\xf4\x18\xf0\xdbl}
(xd\xd7w)\x96R0\xddGRv\xda\xb3\xc6:d\xa0\x82;En\xa9c\xb4\xf0\xca\xe8\xb8\xd3\x1bk\xd20:\x9f\x13L\xfb3j\xb2v\x02\xd3\x92\xaa\xf
L\x81I\xa3\x11\xc9\xc5\xa7\x8eb\xcc9Q\xf7H9\x93\xbc\xbbm\xf1\xcd5\xe1\x93\x03\xc0\xaf\xa5+\xaf\x11Z\x9d\xd7\xfe\xe7\xf1\x1d)E\
\xef\xb7\xadS\x1a\x04J\xae/gg\x81\x99\x18\xe9P\x07{\xde\xc0\xfe\xd5u\xbc\xb3\\$r\xaa\xa6\x95\x05F<1\x9f\xe6k\x96\x8cL\x91\xa0#
([\xc5\xa0X\xedx\xbe\x87|{,
[\x11\x81\xbb6\xd5\xa4{\xf6\xe5\'&h\xf4\xaft\x9d;\xf4\x1d\x05f8\xb0i,x\x9b\x02vX\xd5\x83u\xcf\xca\x9fp\xfeR2\xca\x8a\xbaT\xe1\
\xd4\x82h\x89\x12\x86ni\xdd\xb0\xa3\xd3\xe1J-\xe5O\xf31\x1f,\x00\x17Q\xd2z\x8c\x1aie!
[\xe9\x9eL\xe98m\x19\x03\xf0\x8e\xa0u\xac\xd3T\x84c.\xd9\xce?\xf0\xed\x8c2
f\x10)\x0c\x18\xed\xf0\x1f\xad\x05\xc0\xa7x\xb8\x14!\xb6GW\xcbt>9\xc7\xd6\x9ev\xe5\x14vK\x86,h\xc1\xde\xda6:W\x03rF+;\xc2#F\xe
\x88m@xo\x8d\xce\x7fJ\xc9\xda\x9f\xf5\xac\xf9]\x9bo=\xcd\x0c<\x918j\xcd\x1f\x0c\x937W"
(\x86\x85\xb8\n\x08\x1b\x8e\xee:g~\x9f\x95R\x92\xcc\x97\x91\xc8\xe8\x14\x88\xe6L\x8c{\xc1\xb7\xf8W|9\xccW\xdcMH\xdc\\\x81\xb1#
\xed\xfb\x12\xf6\x91\xbe\x07\xb4\xce\xa0\x81\xba\xa9f\xce\x0f\xce\xb3\xb0\x966\xccT\xe0\xb1e>\xa0\x05=>8\xa7\xbd\x97\xb2\xb7\x
\xbcr2`\x99:\x7f\xb8\xe7\x19\xf5\xeb\xe1DYO*\xdc\xc6\xd6\xc0\xb4\x80o\xa9uj\xda\xba\xbey\xe4\x81\xa5\xbc\x0e$\x0ez\xe0o\xfaoTN
\x99\xa3\xa3\x82\xe2YY,\\\xa9)\xac\x06\xebA\xc4\xf2\xc52\xcct\x97\x94\x14`v\xfen)\xd3\x19Yd\x11\xbc\xcc9\xae\x8b\x99\x8a\x8dD\
{\xa3\xfd\xdec\xa7\x8dq\x0c\xaa\x19\xa3PG\x7fY$\x83\xbd\x17b\xf1\xc1\x7f\x01]\x82\xba\xb0\xd4<ut\xf8W7\xe8\x11\xe9\xf4\x1b\x8b
\x9eB\xde\xce\xaf\x9d\xf5r\xf3\x9f\x9dJ\x9f\x91\xa7\xc5\x1f(\xe2w)\x10\xe2\x88"\x0c\x16U\x19\xef\x00r\x06\xd8\x07~\x94x\x11\xb
b\xec\xed\xf5\x86\x7ff\x89H\x07\xf1w\xb7\xcf\xfe\xdc\xd2\x9e\xca\xb7-U\x91\xdc\xea\xb6\x7fwr\x87\x7f\xb6\xd5\xd9?
\x19l\x88r\xf6\xaam.\xd9dFTf\xd8\x83\x90q\xd79\xf8V\x1a\xd1H\xb9|\x91\xa0H\xb9\xd4q\xe3Z\xc8o4\xda\xdc\xc6GzG=\xec\xf4>
\x9f\x98\xac\xe7\x071\xf3\xae\xce\x85m\x97\x1a\xbf\x15.+\x8c$Ipe\xc3K\xa5\xef\x10\xd44\xe2@N\x93\x9c\xd0\xdcA\x99E\xa2\x86\xd8
[J\xe0\x83\xb0\x1a\x07\x97\xaf\xf3zOg?
&\xeaw$j\xc99\xce\xdf\x88S+\x06\xc4P\x82\xea\xaa\x01\xce\x93\xd4\x85\x1f\xf1I\x1eB\xaf+\x8f\xc6\xd9\xfb\x9e\xa6\xa5\x15\xb6rA6
&D\xef\x0f2\t\xc9\xfc\xaa\xb2\xf4\x84e\xf291!\\r\xba\x02\xc3=
<\xff\x00jQz\x8d\x14Q\xcf\xde\xd2\xf9\xc1\x03c\xf3\xa2\xa2V\x94\x96E\xd5\x80I\xc9\xc6\xd8\xea?
j\x8e\x1a[x\x94\xaa\xb2\x06\xc0\xc8\xc6\x93\xd4\x1f\xce\x96)E\x92\xad\x9c\xc5u\x14i\x14\xb1BP\x84\xef\x1dy-
\xfc\xcdH\xa42\xfbA\xd3\xde1\xe4\x96\x1blG\xf3z\xebDv\x8c\x13:\xc6\xf9Rw\xaa\xb1\xa9&B\x15\x88\x04\x8cn\x00\xf2\xa7\xf62\t\xe1
(\xc2M\xb3g#\x19\xcfZ\x16I\x18\xdb\x0c1Q\x1a\xf4\x1e\x8d\xe1B\xcbq\xce\xd4\xe5A`A\xce6\xe8*\x8a\r\xbb\x0f\x96\x86\xa8\xdf\xd6e(
\xc2\xac\xfb\x8f\xbdJ\xb2\x1e\'\x19\x86=7\x05\x06\x91\xb0\x8f>\x1f\x1a\xf6\xa1\x7f\xd1\xadU\x18\xf3\x93f\xf9\x94\xc4ZP\xdaX\x1
\xf2\xf8\xde\xef\x9do"\xeaQ\x90\x1f\x18
\x01\xb0a\xd3\xa5\x1e\x96\xd9\x92F\x91\x069y8\x8c\x80\xa7\x18\x18\xfa~U\xc3\xdc\xdc\x19\xe3\x8a\xd9\x90\x92t\xb8l\x0c\x8c\x0f\
\x1e\x87b:Q\xb6\xa8c\xb5\x1c\xe2\xeb\x85:H\xc1$\x9f?
\x1cV~)\xd4_\xa9\xe5jWb1\x9e\xa4\x8f\xf8\xa6\xc7\x14\xd4\x92\'\xd4i\xec\xe6\xe6q;\x81#daN\xc7\xbd\xd3\xaf\xad\x0fp\xe2H\xc9\x9
-
\x95.!$\x86\x1c\xc5$\x81\xb1\xde\x8f\xe2Wj\xb6rGi\x9dQLucm\xcf@>\xa2\x92\xd8\xcaR\xe6\x14\x91\xc9\x06A\xa86\xf8\xf8~t\xc3\x88\
\xa4R\x80\n!W|\x15\x18\xd9\xc1\xe9\x9fJ&\xf29\xc0\x16\xd3\xcc\x98\\\xe6\x15\xca\x80z\x9c\xf9\xd1|AV\x1b[Q\x14\xa4)\x8fC\x92\x0
\x9d\x04\xf8\x9a\xf2\xa2\xd6\xcb\xa9*\x15YA\x1a\xda\xca\xf3\\\xaa"\xaa\x8c1\xdb\xa9\xfeb\x8c\x96\xf9\xd1JBQ"9\xef\x1c\x12\xde\
fC\xab\xba\x84j\xff\x00n\xdbVU\xa4\n\xe8\xe9\x90\xaa\xe0\xd6\x9b\x88Fm\xedg\xcc\xa9\xba\xe5UI\xc09\xc6\xdeu\x99h\xdaY\x9e!\xd4
\xc1\x89K\x92\xaew\xee\x1d4M\xc22\xa6m\x16B?
\x13`\x96\x1e\x9e\xb5\x9aqI\xe8\xb6D\xa90Ic`\xd0\x14\x01X\x1c\x16^\x8as\xb7\xda\xb4<*\xd63\xc3"%ZH\xc3HN\xac\xed\xd7\xaf\xcc\x
\xd6\xe3v]\x80\xd8\xef\xd6\x86\xe0\xd2E\x05\xbf4hl\x92\x1d$\'\x1b\x1d\xb1\xbf\xafJ1\xdd\x1eRs\x19O{\xcco\x9d\x89\xeb\xd6\x9a_`
\xae&\x90\xbc\xa28\x83\xa2e\xa4f\xe9\x9f\x1ct\xf1\x15\xe2$\xb2B]\x9f\xbc\x00.7
\xf8\x03\xf3\xcf\xd6\xaaV1LP\x10c\xc7y[\x0cCg\x1dM\x10\xb3\x99\x82B\xe1y\xa4\x8d*\xaau\x11\x9fO\xa9\xf8RpeG\x84\\Op\x82\x13\xa
\xfd\xe9\x97\x0c\x92H/[\x9e@U\x19^\x80\x12\x07\x9f\x81\xfdj\xb8a\x16\xf1\xb3\ry\xf7K\xc5\xf8\x88\xdf\xbd\xb7O\x9dV\x96\xf3\xc8
\xf4\xe4u\xdb\x937\x16\xe2C\x95q\n\xc0\x90(a\x86\x00\x95\xceOO_\xb5,\xe16O\xde\x88\x948!\xb5
\xd5\x81\xe5\xe9\xe5G\xdc\xf0\xab)\xee\xb2\x034\xfas\xa8I\x96#\xc3a\xe3\xd0UPqa\xc3c\x92\x00\xe8\xcf\x925N;\xca\xd8\xd8l1\xe3\
b\xb5\x10\xa2$H\xb1\xb4p\xf5\xc2\x98\xcf_\x95\x1f)c\x8d>\x96\xc5\x8b\xcff^>\r}#\x91(\n\xa5\xb2\x10\x93\xdd\xf8f\x9dX\xf0;8N\xb
{H\xb8}\xb2\x84e\xd6\xccH\x00ns\xd3\xc3\xe3B\xdd^h\x18^ib\xdb\x1d\x19\xf4
}\xfe\xd4\x10\xb4[\xb5\x99\xaeRA\tl\x10\xfb\x14=v\xcf\x8e\x7f:T\xafrz:\x11`r\xdb\xfbL2\x01\x13\xb6NQ\x90gQ\'\xc4\xec:\x1e\xb4-
\xbc\xb2\xf2^\x03o2\x88\xd7$\x90[J\x8f\x1f\xb5?\xb8\x8a\x0bT\xe4\xad\xc6\xa27\x08}\xe0|F\x07\x85#\xf6\xd5\x86\xd2X#\xccBC\xdd]
[t\xc6G\xf3\xc6\xb4B^J\xa8\xa4\xf1\xa8\x95M4f\'i.\x1eWR\x15v\xd8\x91\xe1\xb7\xc6\x86\xbf\xbb\x9a{\x8c\x85\xe5\xa0$\x93\x9c\x0c
)\xee\xf7v\xf4;|\xfe\x95c\xb4\xf2\xf7=0X\x1c`\x0f\xd7\xf6\xab\xa4\x91\'G\n&2+
<\x88\x19\x97Ib\xfd7\x1b\r\xff\x00J\xf5\x97D2\xc6\xad\xac7x\x80\xbf\x95]\xa5\xd1W\x01\x8bi#\n\xbdv\xfc\xe89Cs\xd1\x16&YJ\xeb%s
```



Dataset Structure

```
seg_train/
    buildings
    forest
    glacier
    mountain
    sea
    street

seg_test/
    buildings
    forest
    glacier
    mountain
    sea
    street
```

Step 2: Import Libraries

```
import torch
import torchvision
import numpy as np
import matplotlib.pyplot as plt

from torchvision import transforms, datasets
from torch.utils.data import DataLoader
```

Step 3: Data Augmentation and Preprocessing

```
train_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.RandomHorizontalFlip(),
    transforms.RandomRotation(20),
    transforms.ColorJitter(
        brightness=0.2,
        contrast=0.2,
        saturation=0.2
    ),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])
```

```
test_transform = transforms.Compose([
    transforms.Resize((224,224)),
    transforms.ToTensor(),
    transforms.Normalize(
        mean=[0.485,0.456,0.406],
        std=[0.229,0.224,0.225]
    )
])
```

Step 4: Load Dataset

```
train_dataset = datasets.ImageFolder(
    "seg_train/seg_train",
    transform=train_transform
)
```

```
test_dataset = datasets.ImageFolder(
    "seg_test/seg_test",
    transform=test_transform
)
```

```
class_names = train_dataset.classes
print(class_names)
```

```
['buildings', 'forest', 'glacier', 'mountain', 'sea', 'street']
```

```
train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=32,
    shuffle=False
)
```

Part B: Transfer Learning (10 Marks)

```
import torch.nn as nn
from torchvision import models

device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

model = models.resnet18(pretrained=True)
```

```
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
  warnings.warn(
/usr/local/lib/python3.12/dist-packages/torchvision/models/_utils.py:223: UserWarning: Arguments other than a weight enum or `No
  warnings.warn(msg)
```

```
for param in model.parameters():
    param.requires_grad = False
```

```
num_features = model.fc.in_features

model.fc = nn.Linear(
    num_features,
    6
)

model = model.to(device)
```

```
criterion = nn.CrossEntropyLoss()

optimizer = torch.optim.Adam(
    model.fc.parameters(),
    lr=0.001
)
```

```
train_loss = []
train_acc = []

val_loss = []
val_acc = []
```

```
epochs = 10

for epoch in range(epochs):

    model.train()
```

```
    running_loss = 0
    correct = 0
    total = 0

    for images, labels in train_loader:

        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad()

        outputs = model(images)

        loss = criterion(outputs, labels)

        loss.backward()
        optimizer.step()

        running_loss += loss.item()

        _, predicted = torch.max(outputs,1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    epoch_loss = running_loss/len(train_loader)
    epoch_acc = 100*correct/total

    train_loss.append(epoch_loss)
    train_acc.append(epoch_acc)

    print(
        f"Epoch {epoch+1}/{epochs}, "
        f"Loss={epoch_loss:.4f}, "
        f"Accuracy={epoch_acc:.2f}%"
    )
```

```
Epoch 1/10, Loss=0.6145, Accuracy=79.56%
Epoch 2/10, Loss=0.4013, Accuracy=85.79%
Epoch 3/10, Loss=0.3720, Accuracy=86.85%
Epoch 4/10, Loss=0.3623, Accuracy=86.80%
Epoch 5/10, Loss=0.3520, Accuracy=87.48%
Epoch 6/10, Loss=0.3436, Accuracy=87.34%
Epoch 7/10, Loss=0.3427, Accuracy=87.42%
Epoch 8/10, Loss=0.3341, Accuracy=87.62%
Epoch 9/10, Loss=0.3388, Accuracy=87.52%
Epoch 10/10, Loss=0.3312, Accuracy=87.59%
```

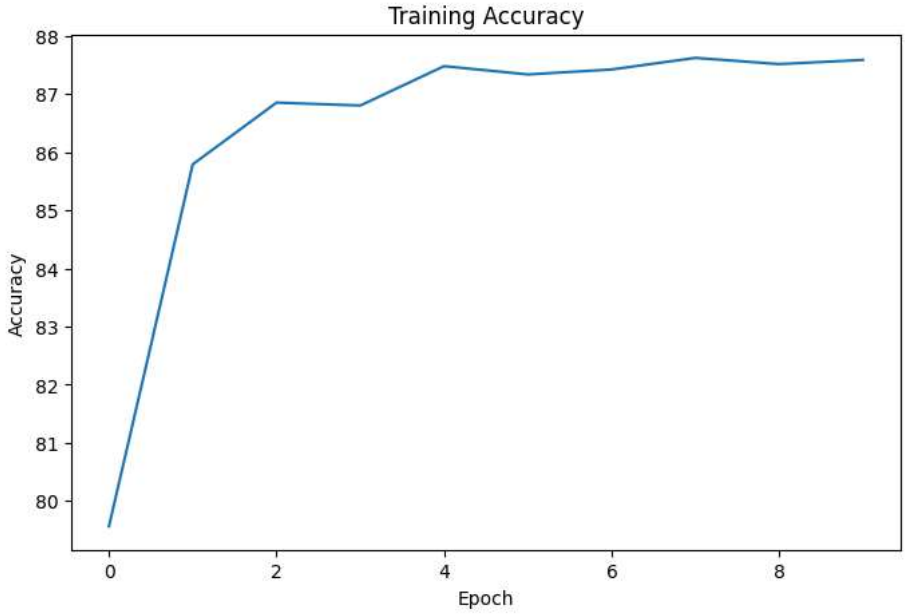
Part C: Model Evaluation and Inference (10 Marks)

```
model.eval()

correct = 0
total = 0

all_preds = []
all_labels = []

with torch.no_grad():

    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs,1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

        all_preds.extend(
            predicted.cpu().numpy()
        )
```

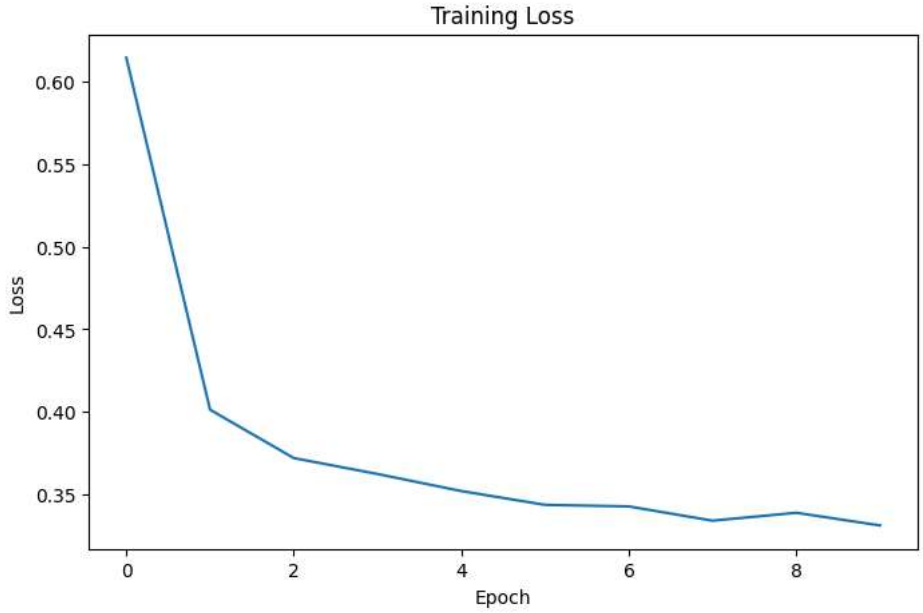
```
        all_labels.extend(
            labels.cpu().numpy()
        )

test_accuracy = 100*correct/total

print("Test Accuracy:", test_accuracy)
```

```
Test Accuracy: 89.86666666666666
```

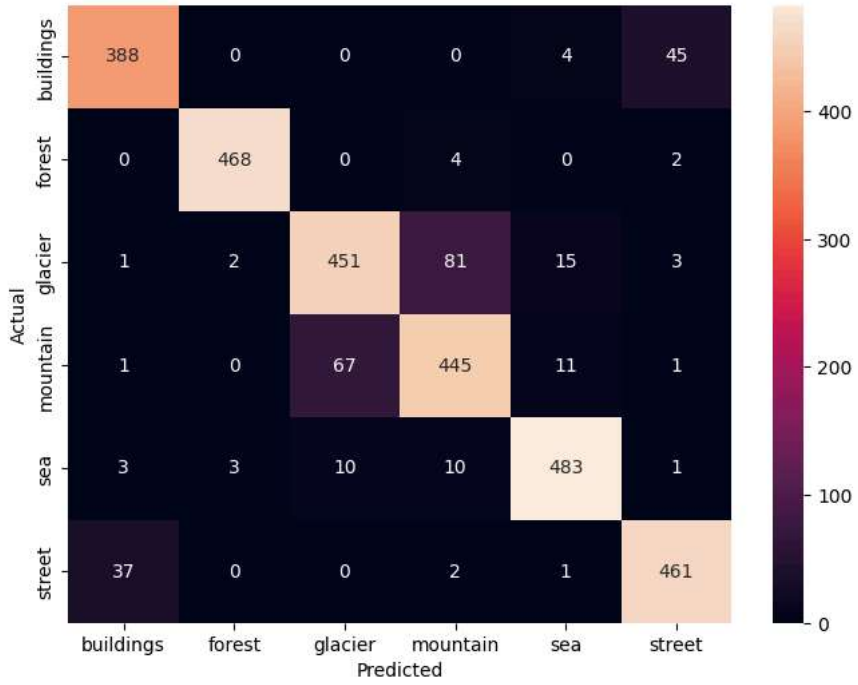
```
plt.figure(figsize=(8,5))
plt.plot(train_acc)
plt.title("Training Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.show()
```




```
plt.figure(figsize=(8,5))
plt.plot(train_loss)
plt.title("Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.show()
```



```
from sklearn.metrics import confusion_matrix
import seaborn as sns

cm = confusion_matrix(
    all_labels,
    all_preds
)

plt.figure(figsize=(8,6))
sns.heatmap(
    cm,
    annot=True,
    fmt='d',
    xticklabels=class_names,
    yticklabels=class_names
)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```




```
from sklearn.metrics import classification_report

print(
    classification_report(
        all_labels,
        all_preds,
        target_names=class_names
    )
)
```

```
              precision    recall  f1-score   support

   buildings       0.90      0.89      0.90       437
      forest       0.99      0.99      0.99       474
     glacier       0.85      0.82      0.83       553
    mountain       0.82      0.85      0.83       525
         sea       0.94      0.95      0.94       510
      street       0.90      0.92      0.91       501

    accuracy                           0.90      3000
   macro avg       0.90      0.90      0.90      3000
weighted avg       0.90      0.90      0.90      3000
```

Detecting Overfitting

Overfitting can be identified when:

- Training accuracy keeps increasing.
- Validation accuracy stops improving.
- Validation loss starts increasing.
- Large gap exists between training and validation accuracy.

Example:

Train Accuracy = 98%
Validation Accuracy = 82%

Solutions:

- Data augmentation
- Dropout
- Early stopping
- Reduce model complexity
- More data

```
def save_model(model, path):
    torch.save(model.state_dict(), path)

save_model(
    model,
    "Q7_trained_model.pth"
)
```

```
def load_model(path):

    model = models.resnet18(pretrained=False)

    num_features = model.fc.in_features

    model.fc = nn.Linear(
        num_features,
        6
    )

    model.load_state_dict(
        torch.load(path)
    )

    model.eval()

    return model
```

```
from PIL import Image

def predict_image(image_path):

    image = Image.open(image_path).convert("RGB")

    image = test_transform(image)
    image = image.unsqueeze(0).to(device)

    model.eval()

    with torch.no_grad():

        outputs = model(image)

        _, pred = torch.max(outputs,1)

    return class_names[pred.item()]
```

```
predict_image('/content/predict_sample.jpg')
```

```
'forest'
```